

Semana 3 — Clase Práctica en Laboratorio

Iteraciones, Sucesiones Numéricas y Aproximación de Constantes

Introducción

Los problemas de esta semana consolidan el uso de **estructuras iterativas** (while, for) junto con las **estructuras condicionales** (if / elif / else) adelantadas en la Semana 2. El eje temático es la exploración de **sucesiones numéricas** y **propiedades de los enteros** mediante algoritmos que requieren bucles con condición de corte no trivial, contadores, acumuladores y detección de ciclos. Cada problema sigue requiriendo la aplicación rigurosa de la **metodología de Pólya** y el **modelo de programa** del curso.

Los seis problemas están organizados en tres bloques con **conexiones deliberadas**: los Problemas 1 y 2 implementan operaciones aritméticas fundamentales (multiplicación y división) exclusivamente mediante sumas, conectando con la aritmética de máquina vista en la clase teórica; los Problemas 3 y 4 exploran sucesiones de enteros con comportamiento impredecible (Collatz, números malabaristas y números felices), ejercitando la iteración con condición de corte dinámica y la detección de patrones; los Problemas 5 y 6 trabajan con propiedades divisorias de los enteros y con la aproximación de constantes matemáticas por series infinitas, respectivamente.

Nota sobre el uso de estructuras iterativas

Las estructuras while y for se presentan formalmente en la Semana 3. En esta práctica se utilizan de manera natural, motivadas por la necesidad de repetir operaciones un número determinado o indeterminado de veces. Sintaxis: while condición: ejecuta el bloque mientras la condición sea verdadera; for variable in range(n): ejecuta el bloque n veces.

Recordatorio: Modelo de programa del curso

Sección Declarativa (docstring): descripción, casos de análisis y prueba, síntesis de casos, recursos (variables con nombre y dominio). Sección Algorítmica: 1) Prólogo (presentación al usuario), 2) Resolución (entrada de datos, cálculos), 3) Epílogo (exhibición de resultados).

Problema 1: Multiplicación de enteros por sumas sucesivas

Enunciado

Escribir un programa en Python que solicite al usuario dos **números enteros** (positivos, negativos o cero) y calcule su **producto** utilizando exclusivamente **sumas sucesivas**, sin usar el operador `*`. El programa debe manejar correctamente los signos y mostrar el resultado.

Fundamento matemático

La multiplicación de dos enteros $a \times b$ puede definirse como la suma de b copias de a (o viceversa):

$$a \times b = a + a + a + \dots + a \quad (b \text{ veces, si } b > 0)$$

Para manejar signos, se aplica la regla de signos: el producto es negativo si exactamente uno de los operandos es negativo. Una estrategia eficiente es trabajar con los **valores absolutos** y determinar el signo por separado. Además, para minimizar las iteraciones, conviene, de sus valores absolutos, sumar el mayor tantas veces como indica el menor:

```
signo = -1 si exactamente uno de los operandos es negativo, +1 si no
abs_a = |a|, abs_b = |b|
mayor = max(abs_a, abs_b), menor = min(abs_a, abs_b)
# Sumar el mayor de (abs_a, abs_b) tantas veces como el menor
producto = 0
repetir menor veces:
    producto = producto + mayor
producto = producto × signo
```

Ejemplos (casos de análisis y prueba)

a	b	Producto	Sumas realizadas	Observación
3	4	12	4+4+4 (3 sumas)	Caso normal, ambos positivos
4	3	12	4+4+4 (3 sumas)	Conmutativo; mismo cálculo óptimo
-3	4	-12	4+4+4 = 12, luego -12	Un operando negativo
3	-4	-12	4+4+4 = 12, luego -12	El otro operando negativo
-3	-4	12	4+4+4 = 12	Ambos negativos → positivo
0	5	0	0 iteraciones	Caso límite: cero
7	0	0	0 iteraciones	Caso límite: cero
0	0	0	0 iteraciones	Caso extremo: ambos cero
1	999	999	999 (1 suma) optimizado	Optimizar eligiendo el menor
-1	1	-1	1 suma	Caso mínimo no trivial

Verificación manual para $a = -3$, $b = 4$

$a = -3$, $b = 4$

```
# Determinación del signo
# a < 0 y b > 0 → exactamente un negativo → signo = -1
abs_a = 3, abs_b = 4

# mayor = max(3, 4) = 4, menor = min(3, 4) = 3
producto = 0
iteración 1: producto = 0 + 4 = 4
iteración 2: producto = 4 + 4 = 8
iteración 3: producto = 8 + 4 = 12
# (3 iteraciones: sumar mayor=4, menor=3 veces)

# Aplicar signo
producto = 12 × (-1) = -12 ✓
```

Orientaciones para la resolución

Análisis: Dos entradas int (a, b). Determinar el signo del resultado. Calcular valores absolutos. Iterar sumando. Aplicar signo. Una salida int. Casos especiales: cero, negativos, optimización por operando menor.

Diseño: Variables: a, b (int, entrada); signo (int, +1 o -1); abs_a, abs_b (int); producto (int, acumulador); i (int, contador). El bucle for i in range(abs_b) es la opción más natural. Para optimizar: si abs_a < abs_b, intercambiar roles.

Codificación: La determinación del signo puede hacerse con: signo = -1 if (a < 0) != (b < 0) else 1, donde el operador != entre booleanos actúa como XOR lógico. Para el valor absoluto: abs() o bien abs_a = -a if a < 0 else a.

Reflexión: multiplicación en la ALU

Las primeras computadoras no tenían instrucciones de multiplicación por hardware. La ALU solo realizaba sumas y desplazamientos (shifts). El algoritmo de multiplicación binaria es exactamente una versión eficiente de sumas sucesivas: en lugar de sumar a exactamente b veces, se examina cada bit de b y se suma a desplazado a la posición correspondiente. Este problema reproduce la lógica elemental que subyace a la operación de multiplicación a nivel de máquina.

Problema 2: División entera y resto por restas sucesivas

Enunciado

Escribir un programa en Python que solicite al usuario un **dividendo** (entero, cualquier signo) y un **divisor** (entero, distinto de cero) y calcule el **cociente entero** y el **resto** utilizando exclusivamente **restas sucesivas**, sin usar los operadores `//` ni `%`. El programa debe verificar que el divisor no sea cero, manejar correctamente los signos y mostrar cociente y resto.

Fundamento matemático

La división entera de a entre b ($b \neq 0$) determina el único par (q, r) tal que:

$$a = b \times q + r, \quad \text{con } 0 \leq r < |b| \quad (\text{definición con resto no negativo})$$

El cociente se obtiene contando cuántas veces se puede restar $|b|$ de $|a|$ antes de que el resultado sea menor que $|b|$:

```
abs_a = |a|, abs_b = |b|
cociente = 0
resto = abs_a
mientras resto >= abs_b:
    resto = resto - abs_b
    cociente = cociente + 1
# Al terminar: abs_a = abs_b × cociente + resto, con 0 ≤ resto < abs_b
```

El signo del cociente sigue la regla de signos. El resto tiene el signo del dividendo (convención truncada, como en C; Python usa convención de redondeo al piso).

Ejemplos (casos de análisis y prueba)

Dividendo	Divisor	Cociente	Resto	Restas	Observación
17	5	3	2	17 : 17→12→7→2	Caso normal
20	4	5	0	20 : 20→16→12→8→4→0	División exacta
3	7	0	3	Ninguna ($3 < 7$)	Dividendo < divisor
-17	5	-3	-2	17 : 17→12→7→2	Dividendo negativo
17	-5	-3	2	17 : 17→12→7→2	Divisor negativo
-17	-5	3	-2	17 : 17→12→7→2	Ambos negativos
0	5	0	0	Ninguna	Dividendo cero
1	1	1	0	1 : 1→0	Caso mínimo
5	0	—	—	—	Error: divisor cero

Verificación manual para dividendo = 17, divisor = 5

```
a = 17, b = 5
# Ambos positivos → signo del cociente = +1
abs_a = 17, abs_b = 5
cociente = 0, resto = 17

iteración 1: resto = 17 - 5 = 12, cociente = 1 (12 >= 5 → continuar)
```

```
iteración 2: resto = 12 - 5 = 7,    cociente = 2    (7 >= 5 → continuar)
iteración 3: resto = 7 - 5 = 2,    cociente = 3    (2 < 5 → terminar)

# Verificación: 5 × 3 + 2 = 17 ✓
# Resultado: cociente = 3, resto = 2
```

Orientaciones para la resolución

Análisis: Dos entradas int (dividendo, divisor). Validar divisor $\neq 0$. Determinar signo del cociente. Bucle de restas sobre valores absolutos. Aplicar signos. Verificar: dividendo == divisor $\times$ cociente + resto.

Diseño: Variables: dividendo, divisor (int, entrada); signo_cociente (int); abs_dividendo, abs_divisor (int); cociente (int, contador); resto (int, acumulador decremental). El bucle es while resto $\geq$ abs_divisor.

Codificación: La validación se hace con if divisor == 0: antes de cualquier cálculo. Al finalizar, verificar con print(dividendo == divisor * cociente + resto) (aquí sí se permite usar * para la verificación, no para el cálculo).

Reflexión: división y restas en la ALU

La división por hardware en procesadores modernos utiliza algoritmos de restauración y no restauración que son versiones optimizadas de la resta sucesiva. En cada ciclo del reloj, la ALU intenta restar el divisor del residuo parcial; si el resultado es negativo, «restaura» (suma de vuelta) o «no restaura» (lo compensa en el siguiente paso). Este problema implementa la versión más simple de ese mecanismo.

Conexión entre los Problemas 1 y 2

La multiplicación por sumas y la división por restas son operaciones inversas implementadas con la misma primitiva: la suma (la resta es suma del opuesto). Verificar esta relación: si $a = 17$ y $b = 5$, el Problema 2 produce $q = 3$ y $r = 2$; luego el Problema 1 puede verificar que $5 \times 3 + 2 = 17$ usando sumas. Esta circularidad refleja cómo la ALU construye la aritmética completa a partir de una sola operación fundamental.

Problema 3: Sucesiones de Collatz y números malabaristas

Enunciado

Escribir un programa en Python que solicite al usuario un **número entero positivo** n ($n \geq 1$) y calcule y muestre:

(a) La **sucesión de granizo** (Conjetura de Collatz) desde n hasta llegar a 1, indicando la **cantidad de pasos** (llamada «tiempo de vuelo») y el **valor máximo** alcanzado (llamada «altitud máxima»).

(b) La **sucesión de números malabaristas** (juggler sequence) desde n hasta llegar a 1, indicando igualmente la cantidad de pasos y el valor máximo alcanzado.

Fundamento matemático

Sucesión de Collatz (granizo)

Propuesta por Lothar Collatz en 1937, se define para cualquier entero positivo n :

Si n es par: $n \rightarrow n // 2$
 Si n es impar: $n \rightarrow 3 \times n + 1$

La **Conjetura de Collatz** afirma que, para todo entero positivo inicial, la sucesión eventualmente llega a 1. A pesar de su simplicidad, permanece sin demostración desde hace casi un siglo y es uno de los problemas abiertos más famosos de la matemática. Se llama «de granizo» porque los valores suben y bajan irregularmente antes de «caer» definitivamente a 1.

Sucesión de números malabaristas (juggler sequence)

Definida por Clifford Pickover, aplica una regla similar pero con potencias fraccionarias:

Si n es par: $n \rightarrow \lfloor n^{(1/2)} \rfloor$ (raíz cuadrada, truncada)
 Si n es impar: $n \rightarrow \lfloor n^{(3/2)} \rfloor$ (n elevado a $3/2$, truncado)

Se conjetura (sin demostración) que toda sucesión de malabaristas converge a 1. En Python, $\lfloor n^{(1/2)} \rfloor$ se calcula con `int(n ** 0.5)` y $\lfloor n^{(3/2)} \rfloor$ con `int(n ** 1.5)`. Para valores grandes, usar `math.isqrt()` (Python 3.8+) mejora la precisión.

Ejemplos (casos de análisis y prueba)

Sucesión de Collatz

n inicial	Sucesión	Pasos	Máximo	Observación
1	1	0	1	Caso base: ya es 1
2	$2 \rightarrow 1$	1	2	Par, un solo paso
3	$3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$	7	16	Crece antes de caer
6	$6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$	8	16	Par, se reduce rápido

n inicial	Sucesión	Pasos	Máximo	Observación
7	$7 \rightarrow 22 \rightarrow 11 \rightarrow 34 \rightarrow 17 \rightarrow 52 \rightarrow 26 \rightarrow \dots \rightarrow 1$	16	52	Tiempo de vuelo largo
27	(muy larga, 111 pasos)	111	9232	Caso famoso: altísima altitud

Sucesión de malabaristas

n inicial	Sucesión	Pasos	Máximo	Observación
1	1	0	1	Caso base
2	$2 \rightarrow 1$	1	2	Par: $\lfloor \sqrt{2} \rfloor = 1$
3	$3 \rightarrow 5 \rightarrow 11 \rightarrow 36 \rightarrow 6 \rightarrow 2 \rightarrow 1$	6	36	Crece rápidamente
4	$4 \rightarrow 2 \rightarrow 1$	2	4	Par: decrece directo
5	$5 \rightarrow 11 \rightarrow 36 \rightarrow 6 \rightarrow 2 \rightarrow 1$	5	36	Igual que parte de $n=3$
9	$9 \rightarrow 27 \rightarrow 140 \rightarrow 11 \rightarrow 36 \rightarrow 6 \rightarrow 2 \rightarrow 1$	7	140	Valores altos intermedios
25	$25 \rightarrow 125 \rightarrow 1397 \rightarrow 52214 \rightarrow \dots \rightarrow 1$	10	52214	Altitud muy alta

Verificación manual para Collatz, $n = 6$

$n = 6$, pasos = 0, maximo = 6

Paso 1: $n=6$ es par $\rightarrow n = 6 // 2 = 3$, pasos = 1, máximo = 6
 Paso 2: $n=3$ es impar $\rightarrow n = 3 \times 3 + 1 = 10$, pasos = 2, máximo = 10
 Paso 3: $n=10$ es par $\rightarrow n = 10 // 2 = 5$, pasos = 3, máximo = 10
 Paso 4: $n=5$ es impar $\rightarrow n = 3 \times 5 + 1 = 16$, pasos = 4, máximo = 16
 Paso 5: $n=16$ es par $\rightarrow n = 16 // 2 = 8$, pasos = 5, máximo = 16
 Paso 6: $n=8$ es par $\rightarrow n = 8 // 2 = 4$, pasos = 6, máximo = 16
 Paso 7: $n=4$ es par $\rightarrow n = 4 // 2 = 2$, pasos = 7, máximo = 16
 Paso 8: $n=2$ es par $\rightarrow n = 2 // 2 = 1$, pasos = 8, máximo = 16

$n = 1 \rightarrow$ fin. Resultado: 8 pasos, altitud máxima = 16 ✓

Verificación manual para malabaristas, $n = 9$

$n = 9$, pasos = 0, maximo = 9

Paso 1: $n=9$ impar $\rightarrow n = \lfloor 9^{(3/2)} \rfloor = \lfloor 27.0 \rfloor = 27$, pasos=1, maximo=27
 Paso 2: $n=27$ impar $\rightarrow n = \lfloor 27^{(3/2)} \rfloor = \lfloor 140.3 \rfloor = 140$, pasos=2, maximo=140
 Paso 3: $n=140$ par $\rightarrow n = \lfloor 140^{(1/2)} \rfloor = \lfloor 11.83 \rfloor = 11$, pasos=3, maximo=140
 Paso 4: $n=11$ impar $\rightarrow n = \lfloor 11^{(3/2)} \rfloor = \lfloor 36.48 \rfloor = 36$, pasos=4, maximo=140
 Paso 5: $n=36$ par $\rightarrow n = \lfloor 36^{(1/2)} \rfloor = \lfloor 6.0 \rfloor = 6$, pasos=5, maximo=140
 Paso 6: $n=6$ par $\rightarrow n = \lfloor 6^{(1/2)} \rfloor = \lfloor 2.449 \rfloor = 2$, pasos=6, maximo=140

Paso 7: $n=2$ par $\rightarrow n = \lfloor 2^{(1/2)} \rfloor = \lfloor 1.414 \rfloor = 1$, pasos=7, maximo=140

$n = 1 \rightarrow$ fin. Resultado: 7 pasos, altitud máxima = 140 ✓

Orientaciones para la resolución

Análisis: Una entrada int ($n \geq 1$). Para cada sucesión: iterar hasta llegar a 1, contando pasos y rastreando el máximo. Caso especial: $n = 1$ (cero pasos).

Diseño: Variables: n (int, se modifica en cada paso); pasos (int, contador); maximo (int). El bucle es while $n \neq 1$. Dentro, if $n \% 2 == 0$: distingue par/impar.

Codificación: Collatz: $n // 2$ y $3 * n + 1$. Malabaristas: $\text{int}(n ** 0.5)$ e $\text{int}(n ** 1.5)$. Imprimir cada valor para visualizar el comportamiento.

Reflexión: conjeturas y computación

La Conjetura de Collatz es un ejemplo de problema que se puede verificar computacionalmente para cualquier caso particular pero no se puede demostrar en general. Se ha verificado para todos los enteros hasta $\sim 2^{68}$ ($\approx 2.95 \times 10^{20}$), pero esto no constituye una demostración. La diferencia entre verificación empírica y demostración formal es uno de los temas centrales de la informática teórica.

Conexión con Arquitectura: paridad y operaciones de bits

Determinar si un número es par o impar ($n \% 2$) equivale a examinar el bit menos significativo de su representación binaria: $0 \rightarrow$ par, $1 \rightarrow$ impar. En lenguaje de máquina, esto se implementa con AND bit a bit ($n \text{ AND } 1$), más rápida que una división. La operación $n // 2$ corresponde a un desplazamiento a la derecha de un bit (shift right).

Problema 4: Números felices

Enunciado

Escribir un programa en Python que solicite al usuario un **número entero positivo** y determine si es un **número feliz**. Si lo es, mostrar la sucesión de sumas de cuadrados de dígitos hasta llegar a 1. Si no lo es, mostrar la sucesión hasta detectar el **ciclo de 8 términos** que caracteriza a los números infelices, e indicar que el número no es feliz.

Fundamento matemático

Un **número feliz** se define mediante el siguiente proceso: dado un entero positivo, se reemplaza el número por la **suma de los cuadrados de sus dígitos** y se repite hasta que el resultado sea 1 (feliz) o hasta que se entre en un ciclo (infeliz).

La operación clave es la **suma de cuadrados de dígitos**:

```
Dado n, extraer dígitos con // y %:
suma = 0
mientras n > 0:
    digito = n % 10
    suma = suma + digito * digito
    n = n // 10
```

Todo número que no es feliz cae eventualmente en el siguiente **ciclo de 8 términos**:

4 → 16 → 37 → 58 → 89 → 145 → 42 → 20 → 4 → ...

Se puede elegir el valor 4 como referente para detectar la existencia de un ciclo, aunque para muchos números infelices sus sucesiones de sumas de cuadrados de dígitos pueden entrar al ciclo de ocho términos por cualquiera de los otros siete valores. El criterio de parada es entonces: **n == 1** (feliz) o **n == 4** (referente de ciclo).

Ejemplos (casos de análisis y prueba)

n	Sucesión	¿Feliz?	Observación
1	1	Sí	Caso base
7	7 → 49 → 97 → 130 → 10 → 1	Sí	Llega a 1 en 5 pasos
10	10 → 1	Sí	Un solo paso
13	13 → 10 → 1	Sí	Dos pasos
19	19 → 82 → 68 → 100 → 1	Sí	Tres pasos
100	100 → 1	Sí	$1^2 + 0^2 + 0^2 = 1$
2	2 → 4	No	Un paso, llega directamente a 4
3	3 → 9 → 81 → 65 → 61 → 37 → 58 → 89 → 145 → 42 → 20 → 4	No	Recorre todo el ciclo hasta llegar a 4
4	4	No	Ya es 4: cero pasos, infeliz
89	89 → 145 → 42 → 20 → 4	No	Miembro del ciclo
999	999 → 243 → 29 → 85 → 89 → 145 → 42 → 20 → 4	No	Recorre parte del ciclo hasta 4

Verificación manual para $n = 19$

$n = 19$

Paso 1: dígitos de 19: 9, 1

$$\text{suma} = 9^2 + 1^2 = 81 + 1 = 82$$

$n = 82$ ($\neq 1$, $\neq 4 \rightarrow$ continuar)

Paso 2: dígitos de 82: 2, 8

$$\text{suma} = 2^2 + 8^2 = 4 + 64 = 68$$

$n = 68$ ($\neq 1$, $\neq 4 \rightarrow$ continuar)

Paso 3: dígitos de 68: 8, 6

$$\text{suma} = 8^2 + 6^2 = 64 + 36 = 100$$

$n = 100$ ($\neq 1$, $\neq 4 \rightarrow$ continuar)

Paso 4: dígitos de 100: 0, 0, 1

$$\text{suma} = 0^2 + 0^2 + 1^2 = 1$$

$n = 1 \rightarrow$ ¡FELIZ! ✓

Verificación manual para $n = 2$ (infeliz)

$n = 2$

Paso 1: dígitos de 2: 2

$$\text{suma} = 2^2 = 4$$

$n = 4 \rightarrow$ ciclo detectado ($n == 4$) $\rightarrow$ NO FELIZ ✓

Orientaciones para la resolución

Análisis: Una entrada int ($n \geq 1$). Bucle externo hasta $n == 1$ o $n == 4$. Bucle interno que extrae dígitos y acumula cuadrados. Salida: sucesión y clasificación.

Diseño: Variables: n (int , valor actual); digito (int , temporal); suma_cuadrados (int , acumulador del bucle interno). El bucle externo es `while n != 1 and n != 4`. El interno extrae dígitos con `n % 10` y `n // 10`.

Codificación: Patrón de **bucles anidados**: el externo gobierna la sucesión y el interno extrae dígitos. La `suma_cuadrados` debe reiniciarse a 0 antes de cada iteración del bucle interno.

Reflexión: detección de ciclos sin memoria

En este problema, la detección del ciclo es simple porque conocemos que 4 es el menor valor del ciclo y por tanto inevitable punto de paso. En general, detectar si una sucesión entra en ciclo requiere técnicas más sofisticadas, como el algoritmo de la tortuga y la liebre de Floyd o el almacenamiento de valores visitados en un conjunto. Estas técnicas se estudiarán en semanas posteriores.

Problema 5: Números perfectos, deficientes y abundantes

Enunciado

Escribir un programa en Python que solicite al usuario un **número entero positivo** n ($n \geq 2$) y determine si es un número **perfecto**, **deficiente** o **abundante**, según la relación entre n y la **suma de sus divisores propios**. El programa debe mostrar los divisores propios encontrados, su suma y la clasificación.

Fundamento matemático

Los **divisores propios** de un entero positivo n son todos los divisores de n menores que n . La clasificación es:

Sea $s(n)$ = suma de divisores propios de n

Si $s(n) == n$: n es PERFECTO
 Si $s(n) < n$: n es DEFICIENTE
 Si $s(n) > n$: n es ABUNDANTE

Para encontrar los divisores propios:

```
suma_divisores = 0
para cada d desde 1 hasta n-1:
    si n % d == 0:
        d es divisor propio de n
        suma_divisores = suma_divisores + d
```

Optimización: basta recorrer desde 1 hasta $\lfloor \sqrt{n} \rfloor$. Si d divide a n y $d \neq n/d$, entonces n/d también es divisor. Esto reduce el bucle de $O(n)$ a $O(\sqrt{n})$.

Los números perfectos son extremadamente raros. Los primeros cuatro son:

Número	Divisores propios	Suma	Fórmula de Euclides
6	1, 2, 3	6	$2^1 \times (2^2 - 1) = 2 \times 3$
28	1, 2, 4, 7, 14	28	$2^2 \times (2^3 - 1) = 4 \times 7$
496	1, 2, 4, 8, 16, 31, 62, 124, 248	496	$2^4 \times (2^5 - 1) = 16 \times 31$
8128	1, 2, 4, ..., 64, 127, ..., 4064	8128	$2^6 \times (2^7 - 1) = 64 \times 127$

Ejemplos (casos de análisis y prueba)

n	Divisores propios	Suma	Clasificación	Observación
2	1	1	Deficiente	Primo: divisor propio = 1
6	1, 2, 3	6	Perfecto	Menor perfecto
10	1, 2, 5	8	Deficiente	$8 < 10$
12	1, 2, 3, 4, 6	16	Abundante	$16 > 12$
28	1, 2, 4, 7, 14	28	Perfecto	Segundo perfecto
1	ninguno	0	Deficiente	Caso límite ($0 < 1$)
13	1	1	Deficiente	Primo grande
18	1, 2, 3, 6, 9	21	Abundante	$21 > 18$
496	1, 2, 4, ..., 248	496	Perfecto	Tercer perfecto

Verificación manual para $n = 28$

```

n = 28
suma_divisores = 0

d=1:  28 % 1 == 0 → sí → suma = 0 + 1 = 1
d=2:  28 % 2 == 0 → sí → suma = 1 + 2 = 3
d=3:  28 % 3 == 1 → no
d=4:  28 % 4 == 0 → sí → suma = 3 + 4 = 7
d=5:  28 % 5 == 3 → no
d=6:  28 % 6 == 4 → no
d=7:  28 % 7 == 0 → sí → suma = 7 + 7 = 14
d=8 a d=13: ninguno divide a 28
d=14: 28 % 14 == 0 → sí → suma = 14 + 14 = 28
d=15 a d=27: ninguno divide a 28

# suma_divisores = 28 == n → PERFECTO ✓

```

Orientaciones para la resolución

Análisis: Una entrada int ($n \geq 1$). Bucle de 1 a $\lfloor \sqrt{n} \rfloor - 1$ que acumula divisores de a pares, y al final se analiza si $\lfloor \sqrt{n} \rfloor$ no es divisor, o si lo es, si determina un divisor (n es un cuadrado) o un par. Comparación final. Salidas: divisores, suma, clasificación.

Diseño: Variables: n (int); d (int, candidato); suma_divisores (int, acumulador); divisores_str (str, cadena de divisores), ultimo (int, último candidato especial). El bucle es `for d in range(2, ultimo):` con `if n % d == 0:` como filtro. La suma de divisores se inicializa en 1, ya que siempre es divisor propio y no es necesario verificarlo.

Codificación: La clasificación final es `if/elif/else` sobre la relación entre suma_divisores y n .

Reflexión: los números perfectos y la historia de la matemática

Los números perfectos fascinaron a los matemáticos desde la antigua Grecia. Euclides demostró que si $2^p - 1$ es primo (un primo de Mersenne), entonces $2^{p-1} \times (2^p - 1)$ es perfecto. Euler demostró que todo número perfecto par tiene esta forma. No se conoce ningún perfecto impar. La búsqueda de primos de Mersenne (GIMPS) es uno de los proyectos de computación distribuida más grandes del mundo.

Problema 6: Aproximación de π por series infinitas

Enunciado

Escribir un programa en Python que solicite al usuario un **número de términos** k ($k \geq 1$) y aproxime el valor de π utilizando dos series distintas, mostrando para cada una: la aproximación obtenida, el valor de referencia (`math.pi`), y el error absoluto. Comparar la velocidad de convergencia.

Fundamento matemático

Serie A: Serie de Leibniz (convergencia lenta)

La serie de Leibniz-Gregory (1674):

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$$

$$\pi/4 = \sum_{i=0}^{\text{hasta } k-1} (-1)^i / (2i + 1)$$

$$\pi \approx 4 \times \sum_{i=0}^{\text{hasta } k-1} (-1)^i / (2i + 1)$$

El signo alterna en cada término: positivo para i par, negativo para i impar. La forma más eficiente de implementarlo es asignar un valor inicial `1.0` a una variable `signo`, calcular cada término como `signo / (2i + 1)` y luego de incorporarlo a la suma cambiar su signo con `signo = -signo`. Esto es más eficiente que `signo = signo * (-1)` (multiplicación innecesaria) y mucho más eficiente que calcular el numerador de cada término como `(-1)**i` (potencia entera recalculada en cada iteración).

La serie converge muy lentamente: se necesitan millones de términos para obtener pocos decimales correctos.

Serie B: Serie de Nilakantha (convergencia rápida)

La serie de Nilakantha (circa 1500, Kerala, India):

$$\pi = 3 + 4/(2 \times 3 \times 4) - 4/(4 \times 5 \times 6) + 4/(6 \times 7 \times 8) - 4/(8 \times 9 \times 10) + \dots$$

$$\pi \approx 3 + \sum_{i=1}^{\text{hasta } k} (-1)^{(i+1)} \times 4 / ((2i)(2i+1)(2i+2))$$

Cada denominador es producto de tres enteros consecutivos. Con solo 10 términos se obtienen 5–6 decimales correctos.

Ejemplos (casos de análisis y prueba)

Serie de Leibniz

k	Aproximación de π	Error absoluto	Observación
1	4.000000	0.858407	Solo primer término
2	2.666667	0.474926	Subestima
5	3.339683	0.198090	Lenta mejora
10	3.041840	0.099753	0 decimales correctos
100	3.131593	0.010000	1 decimal correcto

k	Aproximación de π	Error absoluto	Observación
1000	3.140593	0.001000	2 decimales correctos
10000	3.141493	0.000100	3 decimales correctos
100000	3.141583	0.000010	4 decimales correctos

Serie de Nilakantha

k	Aproximación de π	Error absoluto	Observación
1	3.166667	0.025074	Ya cerca con 1 término
2	3.133333	0.008260	2 decimales correctos
5	3.141839	0.000246	4 decimales correctos
10	3.141591	0.000002	5 decimales correctos
20	3.141593	$\sim 10^{-7}$	6 decimales correctos
50	3.141593	$< 10^{-10}$	Prácticamente exacto

Verificación manual de Leibniz, k = 5

k = 5, suma = 0, signo = 1

i=0: termino = $1/(2 \times 0 + 1) = 1/1 = 1.0$ suma = 1.0, signo = -signo = -1
i=1: termino = $1/(2 \times 1 + 1) = 1/3 \approx 0.333333$ suma = 0.666667, signo = -signo = 1
i=2: termino = $1/(2 \times 2 + 1) = 1/5 = 0.2$ suma = 0.866667, signo = -signo = -1
i=3: termino = $1/(2 \times 3 + 1) = 1/7 \approx 0.142857$ suma = 0.723810, signo = -signo = 1
i=4: termino = $1/(2 \times 4 + 1) = 1/9 \approx 0.111111$ suma = 0.834921, signo = -signo = -1

$\pi \approx 4 \times 0.834921 = 3.339683$

error = $|3.339683 - 3.141593| = 0.198090 \checkmark$

Verificación manual de Nilakantha, k = 3

k = 3, pi = 3.0

i=1: denominador = $2 \times 3 \times 4 = 24$, termino = $4/24 \approx 0.166667$
pi = $3.0 + 0.166667 = 3.166667$ (signo +)

i=2: denominador = $4 \times 5 \times 6 = 120$, termino = $4/120 \approx 0.033333$
pi = $3.166667 - 0.033333 = 3.133333$ (signo -)

i=3: denominador = $6 \times 7 \times 8 = 336$, termino = $4/336 \approx 0.011905$
pi = $3.133333 + 0.011905 = 3.145238$ (signo +)

error = $|3.145238 - 3.141593| = 0.003646 \checkmark$

Orientaciones para la resolución

Análisis: Una entrada int ($k \geq 1$). Dos bucles independientes. Importar `math.pi` para comparación. Tres salidas por serie: aproximación, referencia, error.

Diseño: Leibniz: suma (float), signo (float, ± 1.0), bucle for `i in range(k):`. Nilakantha: pi (float, inicia en 3.0), signo (float), bucle for `i in range(1, k+1):`.

Codificación: Leibniz: `signo / (2*i + 1) → signo = -signo`. Nilakantha: `denominador (2*i) * (2*i+1) * (2*i+2)`. Error: `abs(aprox - math.pi)`.

Reflexión: convergencia y eficiencia algorítmica

Las dos series producen el mismo resultado (π) pero con velocidades radicalmente diferentes. Leibniz necesita ~300.000 términos para 5 decimales; Nilakantha, solo ~10. Esto ilustra un concepto central de la algoritmia: dos algoritmos pueden resolver el mismo problema con eficiencias muy distintas. Aquí la diferencia no es de complejidad temporal (ambos son $O(k)$), sino de la constante de convergencia. Esta idea se formalizará con la notación O grande en semanas posteriores.

Conexión con Arquitectura: precisión de punto flotante

Al acumular muchos términos en una suma de punto flotante, los errores de redondeo se acumulan. Para Leibniz con $k = 1.000.000$, el resultado depende del orden de suma (¡la suma no es asociativa en punto flotante!). Esto conecta con la representación IEEE 754 de la Semana 2: los 52 bits de mantisa del formato double limitan la precisión. Experimentar: ¿cambia el resultado si se suman los términos de menor a mayor valor absoluto?

Notas finales

Estos seis problemas cubren los contenidos prácticos de la **Semana 3** del curso, consolidando **estructuras iterativas** (while, for) junto con las **estructuras condicionales** de la Semana 2. Trabajan con **bucles con condición de corte** (Collatz, malabaristas, números felices), **acumuladores y contadores** (sumas sucesivas, divisores, series), y **bucles anidados** (extracción de dígitos dentro de una sucesión). Los problemas están ordenados en complejidad creciente: Problemas 1–2 con aritmética de máquina, Problemas 3–4 con sucesiones y detección de patrones, Problemas 5–6 con propiedades de enteros y convergencia de series.

Criterios de evaluación formativa

Además de los criterios de las Semanas 1 y 2 (metodología de Pólya, modelo de programa, *literate programming*, análisis exhaustivo de casos), esta semana se evaluarán especialmente: la **correcta elección de la estructura iterativa** (while vs. for según el problema), el **diseño de la condición de corte** del bucle (evitar bucles infinitos), el **uso de acumuladores y contadores** (inicialización, actualización, uso final), y la **traza manual de iteraciones** (verificar paso a paso cada iteración).

Para profundizar

(1) Implementar la multiplicación del Problema 1 usando el **algoritmo de la campesina rusa** (multiplicación por duplicación y mitad), que reduce las sumas de $O(b)$ a $O(\log b)$ y conecta con la multiplicación binaria de la ALU. (2) En el Problema 3, buscar el menor n cuyo tiempo de vuelo de Collatz supere los 500 pasos. (3) En el Problema 4, encontrar todos los números felices menores que 100 y verificar que forman la secuencia OEIS A007770. (4) En el Problema 5, buscar los primeros 4 números perfectos e investigar su relación con los primos de Mersenne. (5) En el Problema 6, investigar y programar la serie de Ramanujan para π (convergencia superexponencial: ~ 8 decimales por término). (6) Combinar los Problemas 1 y 4: implementar la suma de cuadrados de dígitos usando la multiplicación por sumas del Problema 1.

¹ Estos problemas están diseñados para que cada estudiante los resuelva aplicando las cuatro fases de Pólya: Análisis (comprender el problema e identificar casos), Diseño (planificar la solución), Codificación (implementar en Python) y Evaluación (verificar con los casos de prueba). Las orientaciones proporcionadas son guías, no soluciones.